

Unidad 1 (OK)

PDF: "Clase I - Proyectos - Riesgos.pdf".

Proyectos

Proyecto: Es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio.

- Iniciación.
- Planificación.
- Ejecución.
- Seguimiento y Control.
- Cierre.

La gestión de proyectos es la aplicación de conocimientos, habilidades, herramientas y técnicas en las actividades de un proyecto para satisfacer los requisitos del mismo.

El gestor del proyecto (administrador) deberá planear, organizar, asignar personal, dirigir, monitorear, controlar y ejecutar acciones correctivas, innovar y representar (clientes y usuario).

Retos en gestión de proyectos: enfrentar fechas límites, enfrentar limitación de recursos, comunicar efectivamente, conseguir compromisos, establecer hitos medibles, enfrentar cambios y conflictos, negociar vendedores y subcontratistas, etc.

Gestión de Riesgos

Proceso iterativo que se aplica a lo largo de todo el proyecto. Elaborado un plan inicial, se supervisa y al tener más información acerca de riesgos se analizan nuevamente.

Riesgo: Medida de la magnitud de los daños frente a una situación peligrosa.
(Situación peligrosa para el proyecto.)

El riesgo, como la muerte y los impuestos, es una de las pocas cosas inevitables en la vida.

“Mientras que es inútil eliminar el riesgo y cuestionable el poder minimizarlo, es esencial que los riesgos que se tomen sean los riesgos adecuados” – Peter Drucker

Gestión de riesgos: Tarea del director del proyecto que busca anticipar los riesgos que podrían afectar la programación del proyecto o la calidad del software a desarrollar y emprender acciones para evitar riesgos.

La gestión del riesgo es documentar el análisis de los riesgos a lo largo del proyecto junto con el análisis de consecuencias cuando el riesgo ocurra, identificar estos y crear planes para minimizar sus efectos en el proyecto, para el software y para organización.

Categorías de riesgos:

- **Riesgos del proyecto** – Afectan la calendarización o recursos del proyecto
– Personal y presupuesto
- **Riesgos del producto** – Afectan a la calidad o rendimiento del software – Requerimientos
- **Riesgos del negocio** – Afectan a la organización que desarrolla o suministra el software – Marketing, un software útil

Fuentes de riesgo:

- Requisitos. Ambigüedad.
- Estimaciones tiempo y recursos.
- Habilidades personales y grupales.
- Cambios en los requisitos.

Proceso de Gestión de Riesgos

- Identificación de riesgos
- Análisis de riesgos
- Planificación de riesgos
- Supervisión de riesgos

Estrategias frente a riesgo

Estrategias reactivas (ya la cagaron)

- Método:
 - Evaluar las consecuencias del riesgo cuando este ya se ha producido.

- Actuar en consecuencia.
- Consecuencias de una estrategia reactiva
 - Apagado de incendios.
 - Gabinetes de crisis.
 - Se pone el proyecto en peligro.

Estrategias proactivas (para evitar cagarla)

- Método
 - Evaluación previa y sistemática de riesgos.
 - Evaluación de consecuencias.
 - Plan de evitación y minimización de consecuencias.
 - Plan de contingencia.
- Consecuencias
 - Evasión del riesgo.
 - Menor tiempo de reacción.
 - Justificación frente a los superiores.

Identificación de Riesgos

Primera actividad de la Gestión de Riesgos, comprende el **descubrimiento de los posibles riesgos del proyecto**. Se identifican con: Tormenta de ideas, experiencia y lista de posibles riesgos.

Grupos de riesgos:

- **Genéricos:** Son comunes a todos los proyectos.
- **Específicos:** Implican un conocimiento profundo del proyecto.

Categorías:

- Relacionados con el tamaño del producto.
- Con el impacto en la organización.
- Con el tipo de cliente.
- Con la definición del proceso de producción.
- Con el entorno de desarrollo.
- Con la tecnología.
- Con la experiencia y tamaño del equipo.

Análisis de riesgos

Se considera por separado cada riesgo identificado y se decide acerca de la **probabilidad** y **seriedad** del mismo. (se crea una tabla de riesgos). La probabilidad e impacto del riesgo cambian conforme se disponga de mayor información acerca del riesgo y los planes de gestión del mismo se implementen. La tabla de riesgos **debe actualizarse durante CADA ITERACIÓN** del proceso de riesgos.

Planificación de riesgos

El **proceso de planificación** de riesgos considera cada uno de los **riesgos clave** que han sido identificados, así como las estrategias para gestionarlos.

Estrategias:

- **Prevención:** reducir la aparición del riesgo.
- **Minimización:** reducir el impacto del riesgo
- **Plan de contingencia:** estrategia para aplicar ante la aparición del riesgo crítico.

Supervisión de riesgos

La supervisión del riesgo normalmente valora cada uno de los riesgos identificados para decidir si este es más o menos probable si han cambiado sus efectos.

Es un proceso continuo, y en cada revisión del progreso de gestión, cada riesgo clave debe ser considerado y analizado por separado.

Unidad 2 – Validación y Verificación

Introducción

Son actividades encaminadas a determinar si se está construyendo el producto correcto de la manera correcta.

- Se utiliza para mostrar que el sistema se ajusta a su especificación y que cumple con las expectativas del cliente
- Tiene lugar en **todas** las etapas de desarrollo

El objetivo es establecer la seguridad de que el sistema es lo suficientemente bueno para su uso predeterminado.

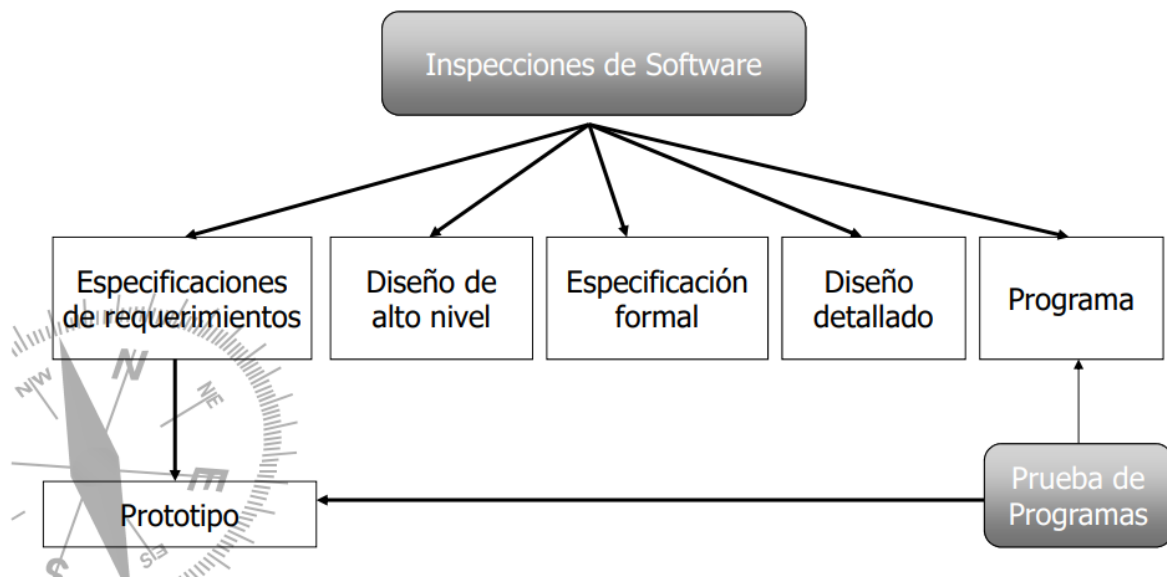
Nivel de confianza:

- La función del software
- Las expectativas del usuario
- El entorno del mercado

Existen dos aproximaciones para el análisis y comprobación de los sistemas:

- **Inspecciones de software:** Analizan y comprueban las representaciones del sistema
- **Pruebas del software:** Implica ejecutar una implementación del software con datos de prueba.

Verificación y Validación



Verificación (concepto)

implica comprobar que el software está de acuerdo con su especificación.

Validación (concepto)

Busca asegurar que el sistema software satisface las expectativas del cliente.
Va más allá de la comprobación de la funcionalidad del sistema.

Verificación (Durante Desarrollo)

¿Estamos construyendo el producto de la manera correcta?

Es un proceso para determinar si los productos de una determinada fase del ciclo cumplen con los requerimientos establecidos en fases previas.

Es desarrollado de manera concurrente con las actividades del desarrollo de software.

Pasos básicos:

- **Inspección:** Revisión técnica a fondo, rigurosa y formal, diseñada para identificar problemas tan cerca de un punto de origen como sea posible.

- **Medición:** Proceso donde se miden la mayor parte de atributos del producto y proceso, para tener información cuantificable para mejora continua.
- **Administración de la configuración:** *conjunto de actividades relacionadas con la administración de la evolución de los productos durante todo su ciclo de vida. (INTERNET)*

Validación (Final Desarrollo)

¿Estamos construyendo el producto correcto? o ¿Construimos el producto correcto?

Es el proceso de evaluar el software al final del proceso de desarrollo para asegurar que cumple con los requerimientos.

Las actividades de validación se dan después de que el software ha sido desarrollado, para determinar si se han implementado correctamente los requerimientos.

La validación comprende actividades dentro de tres procesos básicos:

- Prueba
- Medición, métricas?
- Aumento de la confiabilidad del software: Ayuda en la toma de decisiones cuando hay que dejar de probar y liberar software.

Técnicas de inspección (Sommerville 7ma cap22)

- **Inspección de programas:** Proceso estático, se revisa el software para encontrar errores, omisiones y anomalías. Se puede inspeccionar el código, requerimientos o modelo de diseño.
- Análisis automático del código
- Verificación formal

Pruebas de programas (concepto)

Intentan demostrar que el software es el que el cliente quiere.

Pruebas de defectos (concepto)

Intentan revelar defectos en el sistema en lugar de simular su uso operacional

Depuración (concepto)

- Los procesos de VyV intentan establecer la existencia de defectos en el sistema de software.
- La depuración es el proceso de localizar y corregir los errores
- Luego de la depuración, debe volverse a inspeccionar o hacer las pruebas de regresión.



Unidad 3

PDF 03-IntrVyV-02

Inspecciones de Software.

Son parte del proceso de **verificación** del software.

- Las revisiones o inspecciones de software, son un procesos de V&V estático en el que un sistema se **revisa para encontrar errores, omisiones y anomalías**. (Generalmente se centran en el código fuente).
- Estas inspecciones **se realizan en puntos específicos de la línea de ensamblaje, y se utilizan para asegurar que las partes y los ensamblajes están correctamente contruidos a las especificaciones**.
- La forma más común de inspección sigue siendo el juicio basado en la experiencia.
- Las inspecciones **son usadas para asegurar la calidad** durante el proceso de desarrollo.
- Si se mantiene un registro de los resultados de las inspecciones, se podrá detectar el porcentaje promedio de errores localizados, y entonces indicar cuando un producto está listo para pasar a la siguiente etapa del proceso de desarrollo. (**Estadística**)

Métricas

Una ventaja adicional de las inspecciones es la utilización de métricas a partir de los datos registrados de cada una de ellas. Estas métricas permiten la mejora del proceso de inspección propiamente dicho, como también la mejora del proceso de desarrollo de software en su conjunto. Entre las más destacadas encontramos

- 🔔 Tasa promedio de preparación para inspección.
- 🔔 Tasa de inspección promedio.
- 🔔 Esfuerzo promedio de inspección.
- 🔔 Fallas promedio destacadas por inspección.
- 🔔 Efectividad de la remoción de defectos.

Nivel de Informalidad de inspecciones

-  **Informales:** reuniones donde se discuten aspectos técnicos de algún producto del proceso.
-  **Formales:** programadas como una presentación formal del diseño del software ante un auditorio de clientes, gestores y personal técnico

Inspección Vs. Pruebas

Ventaja	Desventaja
Durante las pruebas, los errores pueden ocultar otros errores.	La inhabilidad de diferenciar un producto de la persona que lo creó. (Se ataca al desarrollador y no al producto (?))
Debido a que la inspección es un proceso estático, no hay que preocuparse de las interacciones entre errores.	Tratar de usar inspecciones sin programar el tiempo adecuado para su preparación y seguimiento
Pueden inspeccionarse versiones incompletas de un sistema sin costos adicionales.	
Puede inspeccionar atributos de calidad más amplios de un programa, como el cumplimiento de estándares, portabilidad y mantenibilidad.	

Proceso de Inspección.

1. **Planificación:** Se asignan los roles de los inspectores. Se distribuyen las copias del producto y material relacionado. En esta oportunidad se planifica la reunión de inspección.
2. **Conferencia Introductoria:** Se presenta el proyecto en desarrollo a los inspectores. Este paso es opcional.

3. **Preparación:** Los inspectores se instruyen individualmente para la reunión de inspección. Estudiando el material aportado en la planificación.
4. **Reunión:** El lector presenta el trabajo mientras los demás investigan el producto en búsqueda de defectos. No debe sobrepasar las 2 horas. Se decide si se acepta o debe retrabajar. Y se obtienen los siguientes resultados:
 - a. Lista de defectos
 - b. Informe gerencia
5. **Re Trabajo:** El autor corrige los defectos detectados en la reunión.
6. **Seguimiento:** Las correcciones realizadas por el autor son chequeadas por el moderador, si éste queda satisfecho queda oficialmente concluida la inspección.

Roles (3 a 6 personas).

- **Moderador:** dirige la reunión de inspección. El más experimentado.
- **Registrador:** es la persona que registra la localización y descripción de los defectos encontrados.
- **Lector:** Es la persona que presenta el producto durante la reunión.
- **Autor:** es la persona que elaboró el producto.

Tipos de revisiones de software

Las más comunes son:

- Auditorias
- Auto-inspección
- Inspecciones técnicas formales (o inspecciones)
- Revisiones
- Recorridos (Walkthroughs)

Auditorias:

Se realizan durante el proceso de desarrollo de un nuevo producto o durante el mantenimiento. Consiste en **asegurar que tanto un producto se ajusta a los planes, políticas, procedimientos y estándares establecidos**. Se llevan a cabo a través de reuniones, observaciones y examinando los productos y procedimientos.

Auto-Inspección

Es una revisión intensa de un producto del trabajo **para asegurar que está correcto, completo, que es consistente y claro**. Es preferible que lo realice alguien que no esté involucrado en el desarrollo del producto y puede involucrar las siguientes tareas.

- Revisión de sintaxis
- Examinar referencias cruzadas
- Violacion de convenios
- Comparación detallada con la especificación
- Lectura de código
- Análisis de diagramas de flujos de control
- Sensibilización de caminos.

Inspecciones técnicas formales.

Se realizan por un equipo que examina un producto determinado, siguiendo roles y un proceso bien definido. El equipo está compuesto por 4 o 5 miembros: moderador, secretario, lector, productor, el agente de V&V, y uno o más expertos.

El proceso normalmente involucra los siguientes pasos:

1. Presentación general
2. Preparación
3. Inspección
4. Retrabajo
5. Seguimiento.

Revisiones

Se enfoca en evaluar un producto en base a guías, estándares y especificaciones de desarrollo y **proveer a los administradores, evidencia de que el proceso se está realizando acorde con los objetivos establecidos**.

La revisión es similar a las inspecciones y recorridos, con excepción de que **incluye a la administración**.

No se enfoca tanto en aspectos técnicos como en las inspecciones.

Recorridos

El objetivo principal es detectar y documentar fallas.

Esto debe ser realizado por todos los involucrados.

Un equipo típico para los recorridos es:

- Coordinador
- Presentador
- Secretario
- Encargado de mantenimiento
- Encargado de estándares
- Agente de acreditación, que representa al usuario
- Revisores adicionales

Diferencias Inspecciones, Recorridos y Revisiones

- Las **inspecciones** son un proceso formal de cinco pasos. Se utiliza una lista de comprobación para descubrir errores. ()
- Un **recorrido** es menos formal, tiene menos pasos, y no usa una lista de comprobación, o documento para reportar el trabajo del equipo.
- Las **revisiones** se enfocan más en deficiencias en el diseño, o desviaciones de los modelos conceptuales o los requerimientos, que en los intrincados detalles de cada línea de la implementación.
- El enfoque de las revisiones no está en fallas técnicas, sino en asegurar que el diseño y el desarrollo concuerdan con las necesidades de la aplicación.

En otras palabras, la revisión se enfoca en el diseño, el recorrido y la inspección en el producto (código), la inspección es más formal.

Unidad 4 – Técnicas de prueba

Introducción

(Incorrecto) Probar es demostrar que no hay errores presentes en el programa

(Incorrecto) El propósito de probar es mostrar que el programa realiza correctamente las funciones esperadas

(Correcto) Probar es el proceso de ejecución de un programa

Objetivos de una prueba

- Descubrir un error
- Un **buen caso** de prueba tiene alta probabilidad de encontrar un error no descubierto.
- Una prueba es **exitosa** si se descubre un error no detectado anteriormente

Principios de las pruebas (Importante)

- A todas las pruebas se les debe poder **hacer un seguimiento** hasta los requerimientos del cliente.
- **Se deben planificar** mucho antes de que empiecen.
- Comenzar por lo pequeño y progresar hacia lo grande
- Pruebas exhaustivas no son posibles
- Deben ser **realizadas por un equipo independiente** para que sean más eficaces.
- **Inspeccionar a conciencia** el resultado de cada prueba, para descubrir posibles síntomas de defectos.
- Cada caso de prueba debe definir el resultado de salida esperado.
- Al generar casos de prueba: incluir datos de entrada válidos y esperados, y no válidos e inesperados.
- Centrado en 2 objetivos:
 - Probar si el software no hace lo que debe hacer
 - Probar si el software hace lo que no debe hacer
- Evitar casos desechables
- No hacer planes de prueba suponiendo que no hay defectos y dedicar pocos recursos
- La experiencia indica que donde hay un defecto hay otros



- Las pruebas son una tarea creativa, como el desarrollo de software

Facilidad de la prueba (Pressman, cap 17 pág. 3, hay mas info)

La facilidad de prueba del software es simplemente la facilidad con la que se puede probar un programa de computadora.

- **Operatividad:** Cuanto mejor funcione, más eficientemente se puede probar
- **Observabilidad:** Lo que ves es lo que pruebas
- **Controlabilidad:** Cuanto mejor podamos controlar el software, más se puede automatizar y optimizar.
- **Capacidad de descomposición:** Controlando el ámbito de las pruebas, podemos aislar más rápidamente los problemas y llevar a cabo mejores pruebas de regresión.
- **Simplicidad:** Cuanto menos haya que probar, más rápidamente podremos probarlo.
- **Estabilidad:** Cuanto menos cambios, menos interrupciones a las pruebas.
- **Facilidad de comprensión:** Cuanta más información tengamos, más inteligentes serán las pruebas.

Características de una prueba (Seguido de lo anterior en el libro)

- Debe tener una alta probabilidad de encontrar un error
- No debe ser redundante
- Debe ser la mejor de su clase
 - En un grupo de pruebas similares, se elige la que tenga más probabilidad de encontrar un error.
- No debe ser ni demasiado sencilla ni demasiado compleja

Enfoques de diseño

Caja blanca

Conocimiento del funcionamiento interno del producto, se prueba que todas las operaciones internas cumplen con las especificaciones.

Garantizan que se ejecuten:

- Por lo menos una vez todos los **caminos independientes** de cada módulo.
- Todas las decisiones lógicas en sus vertientes verdadera o falsa.
- Todos los bucles en sus límites y con sus límites operacionales.
- Las estructuras internas de datos para asegurar su validez.

Criterios de cobertura lógica

De menos riguroso (barato) a más riguroso (caro)

- Cobertura de sentencias
- Cobertura de decisiones
- Cobertura de condiciones
- Criterios de decisión/condición
- Criterio de condición múltiple
- Criterio de cobertura de caminos (impracticable)

Variantes de Caja Blanca

- Prueba de camino básico
- Prueba de condición
- Prueba de flujo de datos
- Prueba de bucles

Prueba de camino básico

Complejidad ciclomática (Complejidad de McCabe $V(G)$)

- Popular en el diseño de pruebas
- Es un indicador del número de caminos independientes que existen en un grafo
- Fórmula
 - $V(G) = a - n + 2$
 - $V(G) = r$
 - $V(G) = c + 1$
- Donde:
 - a: Cant. de arcos o aristas del grafo.
 - n: Cant. de nodos.
 - r: Cant. de regiones cerradas del grafo.
 - c: Cant. de nodos de condición.

- $V(G)$ marca el límite **mínimo** de casos de prueba para un programa.
- Cuando es superior a 10, se debe considerar dividir en módulos.

Derivación a casos de prueba

- Dibujar gráficos de flujos con el código
- Determinar $V(G)$
- Determinar conjunto básico de caminos independientes.
- Preparar los casos de prueba
 - Variable
 - Entrada: Dato de prueba
 - Salida: Resultado esperado

Prueba de condición (?????) Pressman cap17 pag 11

Expresión simple: $E1 <\text{operador}> E2$

- $E1$ y $E2$ operaciones aritméticas
- $<\text{operador}>$: $<, <=, =, >, >=, >$

Expresión compuesta: Dos o más operadores, booleanos, paréntesis.

- La cobertura de la prueba de una condición es sencilla
- La cobertura de la prueba de las condiciones de un programa da una orientación para generar pruebas adicionales del programa.

Prueba de bucles

Bucles simples

- Omitir por completo el bucle
- Solo un paso por el bucle
- Dos pasos por el bucle
- m pasos, donde $m < n$
- $n=1$; n , $n+1$ pasos por el bucle ??????????????????????????????

Bucles anidados

- Iniciar el bucle más interno, asignar a todos los bucles mínimos
- Aplicar pruebas de bucles simples al más interno. a los externos valores mínimos.
- Trabajar hacia fuera.

- Externos con valores mínimos, anidados con valores típicos.

Bucles concatenados

- Independientes: Bucles simples
- No independientes: Bucles anidados

Bucles no estructurados

- Rediseñar

Caja negra

Se conoce la función y se aplican pruebas para demostrar que cada función es plenamente funcional.

Intenta encontrar errores en la siguientes categorías:

- Funciones incorrectas o ausentes
- Errores de interfaz
- Errores en estructuras de datos o acceso a bases de datos externas
- Errores de rendimiento
- Errores de inicialización y terminación.

Variantes de Caja Negra

- Métodos de prueba basados en grafos
- Partición equivalente
- Análisis de valores límite
- Prueba de Comparación
- Conjetura de errores

Pasos a seguir

1. Entender los objetos que se van a modelar y las relaciones que conectan a estos
2. Definir una serie de pruebas que verifique que todos los objetos tienen entre ellos las relaciones esperadas.

Basado en Grafos

Se crea un grafo de objetos y sus relaciones. Se deben entender los objetos y las relaciones entre estos.

Se diseñan pruebas que cubran todos los objetos y relaciones para descubrir errores.

Partición de equivalencia

- Divide el dominio de entrada de un programa en clases de datos a partir de los cuales pueden derivarse a casos de prueba
- Una clase de equivalencia representa un conjunto de estados validados y no validados para las condiciones de entrada
 - Si una condición de entrada especifica un **rango**, se define una clase de equivalencia válida y dos no válidas.
 - Si una condición de entrada requiere un **valor específico**, se define una clase de equivalencia válida y dos no válidas.
 - Si una condición de entrada especifica un **miembro de un conjunto**, se define una clase de equivalencia válida y otra no válida.
 - Si una condición de entrada es **booleana**, se define una clase de equivalencia válida y otra no válida.

Análisis de valores límites

Reglas para identificar clases

1. Si una condición de entrada especifica un rango entre a y b, los casos de prueba deben diseñarse para estos valores y los que se encuentran apenas abajo y apenas arriba.
2. Si una condición de entrada especifica valores, deben diseñarse casos de prueba para máximo y mínimo, y apenas abajo y apenas arriba.
3. Aplicar 1 y 2 para las condiciones de salida.
4. Diseñar casos de prueba para los valores límites de las estructuras de datos.

Unidad 5 – Pruebas orientada a objetos

PDF: “PruebasOO.pdf”

Comienzan a hacerse en el análisis y diseño.

Todos los modelos orientados a objetos deben ser probados (revisiones técnicas formales), para asegurar la exactitud, completitud y consistencia dentro del contexto de la sintaxis, semántica y pragmática del modelo.

Estrategia: Comenzar a “probar lo pequeño” y funciona hacia fuera, “hacia lo grande”.

Pruebas de unidad

La unidad mínima de prueba es **la clase** (Nombre, atributos, métodos.)

Las pruebas de clases se conducen mediante las operaciones encapsuladas en la clase y su comportamiento.

Casos de prueba

Cada caso de prueba debe ser identificado separadamente, y explícitamente asociado con la clase a probar.

Debe declararse el propósito de la prueba.

Debe desarrollarse una lista de pasos a seguir.

Secuencia de operaciones a ejecutar.

Pruebas a nivel de clases:

Establecer secuencias de pruebas al azar

Técnica: Particiones a nivel de clases:

1. Cada atributo se relaciona con solo con un conjunto de las operaciones de la clase.
 2. Reduce el número de casos de prueba para validar la clase
 3. Criterio: Partición basada en estados
 4. Diagrama de transición de estados
- Técnica
 - Separar las clases en partes (rebanadas): para cada atributo identificar las operaciones relacionadas a este
 - Generar rebanadas:
 - **Grafo de llamadas**: Conjunto de árboles donde la raíz es un método y las hojas son los atributos a que hace referencia y métodos que invoca.



- **Matriz de uso mínima:** Una fila por cada atributo; una columna por cada método; las celdas según la relación entre atributo-método.

Permite comprobar cohesión: descomposición excesiva; revisar diseño.

- **T** si el método modifica el valor del atributo
 - **R** si el método informa (consulta) el valor del atributo
 - **O** si el método utiliza el valor del atributo en alguna operación
 - **V**acío si no hay relación
- Secuencia de métodos para la prueba
 - Se categorizan los métodos: Constructores; reportes; transformadores; otros.
 - Se realizan las pruebas para los métodos de reportes (r)
 - Se prueban los constructores
 - Se prueban los métodos de transformaciones (t y o)
 - Se prueban otros métodos.
- Generación de los casos de prueba

Prueba de los módulos transformadores

- Se elige un atributo y los transformadores que corresponden a su "rebanada". Colocando al reportero al final.
- **Se reforman las permutaciones de los transformadores, quedando el reportero siempre en el final**
- Se consideran varios casos de prueba para cada secuencia

Herencia

- Probar la clase base y las subclases (derivadas) con la estrategia explicada
- En la herencia se presenta:
 - Métodos nuevos (o redefinidos) que afectan atributos de la clase base
 - Métodos de la clase base que afectan atributos de la clase base
 - Métodos de la clase base que afectan atributos de la clase derivada
 - Métodos nuevos que afectan a atributos nuevos.

Pruebas de integración

Para encontrar:

- Problemas de configuración
- Funciones faltantes, o con conflictos
- Uso incorrecto o inconsistentes de base de datos
- Violación a la integridad de datos
- Llamadas a métodos equivocados
- Parámetros erróneos
- Problemas del manejo de la Máquina Virtual
- Recursos insuficientes

Unidad mínima de funcionalidad: **Casos de uso**

Se elaboran diagramas:

- Colaboración/secuencia que representan los escenarios para una misma situación:
 - Objetos que colaboran.
 - Interacciones entre objetos. Secuencia de mensajes.

Toma importancia todos los componentes a integrar.

De estos componentes interesa su **estado expresado en proposiciones lógicas**.

Tipo de elemento	Posibles estados
objeto	Existe, no existe, existe pero es de versión inadecuada
Recurso compartido (archivo, BD)	Existe, No existe, No es accesible (falta autorización, exceso de usuarios, bloqueado)
Dispositivo (red, impresora)	Listo, en problemas, desconectado

Estado de los objetos involucrados: **Valores de los parámetros**.

Los **valores que pueden recibir los métodos invocados aumentan los casos de prueba**

Generar los casos de prueba:

- Para cada diagrama de secuencia, generar los estados posibles de los elementos involucrados.
- Para cada método en las interacciones incluidas en el diagrama de secuencia determinar los conjuntos de valores adecuados.

- Con cada estado y cada representante de conjunto de datos se forma un caso de prueba, combinando los diferentes valores de cada categoría.

Unidad 6 Parte 1 – Pruebas de Validación y Sistemas

Testing de Integración

Verificar que la interacción entre unidades y sus interfaces (las cuales suponen correctas) es correcta.

- Casos de test seleccionados para testear interfaces en lugar de funcionalidad de unidad.
- Cerca del 40% de los errores son problemas de interfaces e integración.
- Descubrirlos temprano reduce costos de corrección.
- Mejor testeado por desarrolladores, conocen las interfaces.
- El tester de funcionalidad está muy motivado para eliminar sistemáticamente los errores de interfaces.

Estrategias:

¹ **Drivers:** Son módulos hechos para simular el comportamiento de los módulos de orden superior que llaman al testeado.

² **Stubs:** Son módulos de orden inferior que simulan las funciones que el módulo testeado puede solicitar.

Test Big-Bang:

- Cada unidad es testeada por separado, luego se integran y se testea completo.
- Ahorra tiempo en el proceso de Testing de Integración.
- Si los casos de test y sus resultados no están bien registrados, el proceso es complicado y perjudica el objetivo del Testing de Integración.

Test Incremental:

- Bottom-Up:
 - Las unidades se testean de abajo hacia arriba en el grafo de llamadas. Primero las unidades terminales aisladas.
 - Unidades **driver**¹ para unidades de nivel superior.
 - Se reemplazan los **drivers**¹ por unidades de siguiente nivel.
 - Módulos críticos en niveles inferiores.

- Planeamiento de programación.
- Top-Down:
 - Comienza desde el tope.
 - Utiliza unidades **stubs**² para simular a las que llamaría.
 - Se reemplazan los **stubs**² por unidades reales y se continúa hacia unidades terminales.
 - Permite tener una visión preliminar del sistema (sim. prototipo evolutivo).
 - Seguimiento fácil y mayor aceptación.
- Variaciones:
 - Diseñar, Codificar y Verificar un nivel por vez.
 - Enfoque Zig-Zag.
 - Finalizar el diseño antes de codificar.
 - Enfoques mixtos.

Errores que se desea encontrar

De interpretación:

El comportamiento esperado por el usuario de un módulo no coincide con su funcionalidad.

- Función errónea: Función de B no es requerida por A.
- Función Extra: Algunas funciones de B no son requeridas por A.
- Función Perdida: Algunos parámetros de A hacia B que están fuera del dominio de B.

Llamada errónea

La llamada se coloca en un lugar equivocado en el módulo llamador.

- Instrucción de llamada extra: La instrucción está en un camino equivocado.
- Ubicación equivocada: La instrucción está en el lugar equivocado del camino correcto.
- Instrucción pérdida: Falta la instrucción.

De Interfaz

La interfaz estándar tiene errores. *Por ejemplo tipo de datos de los parámetros*

Puntos claves para la integración

- Conectar de a poco las partes más complejas, para identificar más fácil causas de errores.
- Minimiza la programación auxiliar (drivers/stubs), para ahorrar el esfuerzo de programación.

Puntos claves Testing de Integración

- Aplicar el criterio de Condición Límite.
- Probar todos los parámetros de punteros con nulos.
- Diseñar tests que hagan fallar al componente.
- En el caso de memoria compartida, cambiar la secuencia de acceso a los datos.
- Usar testing de estrés en interfaces de mensajes.
- En interfaces de mensajes, cambiar el orden.

Testing de Sistema

Su objetivo es chequear que la colección de elementos constituyentes como un todo, se comporte apropiadamente (**asumiendo que sus componentes ya lo hacen individualmente**).

Tests

Funcionalidad

Se verifica que cada una de las funciones especificadas en el documento de requerimientos estén cubiertas y se ejecuten correctamente.

Performance

Test de cualidades que no pueden definirse a nivel de módulos, sino que dependen del sistema global.

- Puede medirse en términos de tiempo de respuesta o utilización y probarlo bajo diferentes combinaciones.
- Rendimiento del peor caso.
- Rendimiento promedio.

Seguridad

Se aplican tests ordinarios para confirmar que la seguridad del sistema no está comprometida.

Configuración

Probar bajo diferentes combinaciones de hardware y software. Las conexiones físicas con las que interactúa el sistema pueden ser reemplazadas por diferentes razones.

Robustez

Testear el sistema bajo condiciones inesperadas.

Stress

Situaciones de recursos saturados. Intenta romper el sistema. Condiciones límites, distorsión del orden de procesamiento, aumenta las acciones simultáneas, etc.

- Testing de carga: Simular el efecto de muchos usuarios utilizando el sistema en forma concurrente.
- Testing de volumen: Simular la entrada de grandes volúmenes de datos.

Confiabilidad

Medición de la probabilidad de ocurrencias de fallas.

- Relacionado a tolerancias a fallas.
- Permite disponibilidad.
- Promedio Cantidad de fallas.
- Cantidad de fallas por unidad de tiempo.
- Tiempo medio entre fallas.

Usabilidad

El sistema es diseñado e implementado para ser usado, la usabilidad se define como de apropiado, funcional y efectiva es la interacción con el usuario.

- Los requerimientos de usabilidad son generalmente pobres y difíciles de describir lo que lleva a un testeo también pobre.
- En general no se puede realizar este test hasta muy tarde en el ciclo de vida del software
- En esta etapa un error o cambio puede ser muy caro.

- Testeo de Interfaces de Usuario.

Testing de Regresión

Se define como la repetición selectiva de un conjunto de test o *Test suite* previamente ejercido sobre un sistema o unidad, para comprobar que las modificaciones no hayan causado efectos laterales.

- Se realiza para ver si un sistema modificado funciona al menos tan mal como antes de la modificación.
- Se deben volver a ejecutar en lo posible todos los test ya hechos cada vez que se modifican los requerimientos, test, plataformas, etc-
- Se intenta establecer una base mínima de corrección y para evitar que el proceso se des controle.
- Se debe testear la especificación modificada contra el programa modificado.

Regresión Progresiva:

- Surge cuando la especificación es modificada
- Si surgen nuevas mejoras o nuevos requerimientos de datos la especificación debe modificarse
- Nuevos módulos van a ser agregados al sistema
- Se debe testear la especificación modificada contra el programa modificado

Regresión Correctiva:

- No cambia la especificación
- Solo cambian algunas instrucciones y posiblemente algunas decisiones de diseño.
- Muchos casos de test en el plan de testeo anterior van a seguir siendo válidos.
- Pero otros no seguirán siendo válidos.

Tipos de Casos de Test

Casos de Test Reusables

- Incluyen ambos tipos de casos de test (Correctiva y progresiva)
- Testean las partes no modificadas de la especificación y su correspondiente programa no modificado.
- No necesitan ser re ejecutados ya que obtendrán los mismos resultados que antes.

Casos de Test re-testeables

- Otra vez incluyen ambos tipos de test. *(Creo que es Correctivo nomas, porque no cambiar la especificación)*
- Deben repetirse porque el programa que es testeado cambió.
- Aunque la especificación no lo hizo.

Casos de Test Obsoletos

- Aquellos test que deben ser descartados porque ya no siguen siendo válidos por los cambios en la especificación.
- Aquellos test sobre los programas que ya no controlan las estructuras deseadas debido a los cambios en el mismo.

Casos estructurales nuevos

- Casos basados en la estructura que incrementen el cubrimiento estructural de un programa.

Casos nuevos para la especificación.

- Incluye solo los casos de test basados en la especificación.
- Testean el nuevo código generado a partir de la parte modificada de una especificación

Testing de Aceptación

- Implica Validación del sistema
- Realizado por el usuario final o cliente para verificar conformidad con el producto desarrollado

- Está basado en los requerimientos del usuario.
- Está dirigido a demostrar que el sistema cumple con los requerimientos solicitados.

Versiones

Alpha

- No posee todas las funcionalidades, solo centrales y está capacitado para recibir entradas y generar salidas.
- No está pensado para estar en funcionamiento.
- Se realiza con representantes del cliente, en el entorno de la empresa que desarrolla el software.
- Realizado por desarrolladores.

Beta

- Directamente por los usuarios finales en su entorno.
- El proceso de entregar una versión beta a los usuarios se llama beta release.
- Al sistema en esta etapa se lo denomina como prototipo, acceso temprano, preview, etc.

Beta Testers

- Contratados para probar el producto.
- Deben reproducir el entorno de los usuarios.
- Trabajan sobre un producto por horas, repitiendo las mismas acciones para intentar encontrar fallas.
- Beta testers GNU: para software libre, donan su tiempo.
- Free Beta Trials: Oportunidad de probar productos nuevos antes de llegar al mercado. Testing gratuito y posicionamiento temprano en el mercado.

Release Candidate

- Versión con potencial para ser un producto final.
- Listo para salir, a no ser que se encuentren fallas fatales.
- Debe poseer todas las funcionalidades.

Gold

- Sistema en producción final.
- Casi idéntico a la versión anterior, pero sin fallas.
- Considerado estable y relativamente libre de errores.
- Posee la cantidad suficiente para su amplia distribución y uso por usuarios finales.

Unidad 6 Parte 2 – Pruebas de Validación y Sistemas

Testing de Integración OO

Una estrategia de integración debe responder:

- ¿Cuáles componentes son el centro del test de integración?
- ¿En qué secuencia serán ejercitadas las interfaces de los componentes?
- ¿Cuáles técnicas de testing deberían usarse en cada interfaz?

En el desarrollo OO el testing comienza:

- Dentro de una clase: Integrando sus métodos.
- Dentro de una jerarquía de clases: Integrando métodos de las superclases en las subclases.
- Dentro de un cluster de clases relacionadas: Integrando las colaboraciones entre las clases.

Tipo de errores detectados:

- Funciones perdidas, solapadas o erróneas
- Problemas de versión
- Violaciones de la integridad de los datos de una BD
- Llamada incorrecta a un método debido a errores en el código o parámetros
- Llamadas a métodos inexistentes (poseen el mismo nombre pero no los mismos parámetros)
- Uso incorrecto de la máquina virtual
- Llamada a objetos desasignados

A nivel de Clase:

Los componentes que se integran son:

- Métodos de la clase.
- Métodos de la superclase.
- Variables de instancia.
- Parámetros enviados y recibidos.

Tipos de Test:

- Integración Intraclase Bottom-Up:
 - Se centra en el uso de las dependencias.
 - Los métodos son testeados en un orden específico.
 - Constructores, get, set, métodos boolean, iteradores y destructores.
- Integración Intraclase Big Bang:
 - Se testean todos los elementos al mismo tiempo.
 - Útil si: La clase es pequeña y simple y existen pocas dependencias intraclases.
- Testeo de Servidores Polimórficos:
 - Cada método polimórfico es ejecutado tanto en la clase que lo define como en las subclases que lo heredan.
 - Diferente al test Mensaje Polimórfico (Unidad) que testea métodos polimórficos dentro de un método llamador.
 - Se implementa un driver que posea todas las posibles llamadas polimórficas.

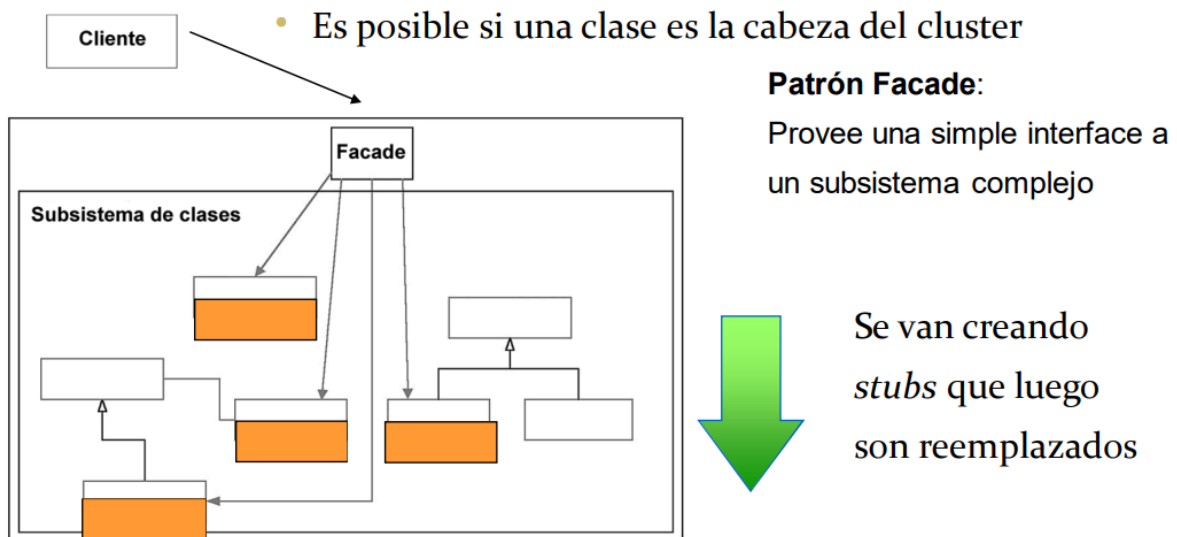
A nivel de Cluster

Los componentes que se integran son:

- Objetos servidores usados por la clases bajo testeo
- Mensajes Interclase

Tipos de test posibles

1. Integración Bottom-up.
 - a. Es el más usado a este nivel.
2. Integración Top-down
 - a. Es posible si una clase es la cabeza del cluster



3. Test Big-Bang

- a. No es recomendable
- b. Solo si unos pocos componentes nuevos son agregados a un sistema estable

A nivel de Subsistema/Sistema:

La meta es lograr un sistema estable.

Los componentes que se integran son: Cluster, Subsistemas y Mensajes Interpaquete.

Tipos de Test

- Integración Bottom-Up.
- Integración Top-Down.
- Integración Big Bang:
 - Recomendable solo para arquitecturas monolíticas.
 - Los componentes están tan acoplados que no pueden testearse en forma separada.
- Integración de colaboración.
- Dependen de la arquitectura:
 - Integración Cliente/Servidor.
 - Integración por Capas.

Testing de Sistemas OO

Tipos de test posibles

1. Testeo de configuración y compatibilidad
2. Testeo de Performance
3. Testeo de Integridad y Tolerancia a Fallos
 - a. Testeo de Stress
 - b. Testeo de Concurrencia
 - c. Testeo de Reinicio/Recuperación

3. Testeo de Integridad y Tolerancia a Fallos.

- Testeo de Concurrencia
 - Simula configuraciones típicas de ejecución concurrente y niveles típicos de carga
 - Intenta que el sistema termine anormalmente
- Testeo de Reinicio/Recuperación
 - Simula las posibles causas de fallas en el sistema y verifica cómo se recupera
 - Cortes de luz, terminaciones anormales
 - Transacción de actualización diaria de BD corriendo por 15hs. Un evento determinado hace que el sistema falle
 - El reinicio, debe hacerse desde el último punto de commit? O debe rehacerse TODA la transacción?
- Testing de Aceptación
 - Alpha → Beta → Release Candidate → Gold

Testing de Regresión OO

- Análisis de Impacto de Cambio.
- Se intenta reducir el esfuerzo de test.
- Seleccionamos entre diferentes estrategias de integración.
- Se aíslan de forma precisa los efectos de los cambios en partes no modificadas de nuestro sistema.

Se realiza cuando

- Se desarrolló una nueva clase:
 - Se vuelven a realizar los test de la superclase en la subclase.
 - Se crean nuevos test para la subclase.
- Se modificó una superclase:
 - Se vuelven a realizar los test sobre la superclase en la subclase y sobre la subclase.
- Se modificó una clase servidora:
 - Se vuelven a realizar los test para los clientes de la clase modificada.
 - Se testea la clase modificada.
- Se soluciona un error (bug):
 - Si el error fue revelado por un test, volver a ejecutarlo.
 - Volver a realizar los test sobre todas las partes que dependan del código cambiado para verificar efectos secundarios del error.
- Se genera un nuevo incremento para el Testing de Sistema o de Integración:
 - Todos los test previamente definidos para la etapa incremental anterior deben ser realizados nuevamente.
 - Si podemos correr los nuevos test que consideran los elementos agregados.
- Se estabilizó un sistema y se generó la última entrega (release):
 - Volver a realizar los test del sistema completo para detectar bugs omitidos.

Los casos de test siguientes deben ser descartados o rediseñados.

Casos de Test Averiados:

- El caso de test falla.
- Usan interfaces, segmentos de código, estructuras de datos que han cambiado.

Casos de Test Incontrolables

- Generan estados o salidas imprevisibles

Casos de Test Redundantes

- Dos o más test son redundantes si sus entradas y salidas son idénticas
- Dos casos de test idénticos no son necesariamente redundantes.



FATAL ERROR

- © 2020 - 2022, KochCorp

Unidad 7 – Indicadores de calidad

PDF: “IndicadoresCalidad2020.pdf”

Los líderes de proyectos tenemos que evitar tomar medidas a “ojo”. Para eso tomamos decisiones usando diferentes herramientas basadas en evidencia física.

Una posible evidencia física que sirve como base para la registración de avance puede ser una porción de software construida. ¿Cuál es la evidencia?. El producto **desarrollado, probado y estabilizado** (sin defectos críticos).

Tipos de avance:

- **Avance por calendario:** Se mide avance solo por paso del tiempo.
- **Avance por código completo:** Se mide el avance cuando una porción del código está completa, pero sin saber el grado de estabilidad que tiene.

Indicador de Funcionalidad Completa

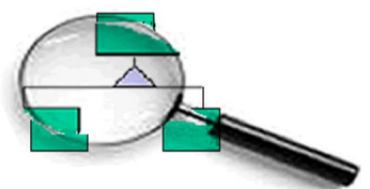
Mide avance cuando una funcionalidad está completa, pero...

No hay avance si la funcionalidad no está completa, o sea que no está desarrollada, probada y estabilizada.

1. Determinar las funcionalidades
2. Asignar un peso a cada funcionalidad
3. Estimar la fecha en que la funcionalidad estará completa
4. A medida que avanza el proyecto registra las fechas reales de funcionalidad completa

Consideraciones:

- **Validez:** Este indicador solo es válido en etapa de construcción.
- **Proceso:** Esté indicado no generará avance si el equipo no se focaliza en cerrar temas.



- **Síndrome del 0%:** Si se considera completa una funcionalidad cuando no tiene ningún defecto, evitaremos el síndrome del 90%, pero caeremos en el síndrome del 0%.
- **Funcionalidad = código:** Es su forma más pura, este indicador sólo considera funcionalidad al producto final para el usuario.

Indicador de Nivel de Calidad

El indicador de funcionalidad completa divide al producto en dos estados: funcionalidad **completa** o **no completa**.

Algunas veces necesitamos utilizar estados intermedios.

¿En qué fecha terminamos?

Con el indicador de funcionalidad completa podemos obtener una tendencia.

En la etapa de estabilización comienza cuando el indicador de funcionalidad completa deja de tener validez.

Solo resta corregir los defectos remanentes y encontrar los que no encontramos aún.

Indicador de Evolución de Prueba

Se utiliza para obtener una tendencia y poder predecir la fecha de finalización

Mide los defectos, cuantos aparecen y cuantos se cierra. (por día, semana, etc...)

Proceso:

- Registrar defectos encontrados
- Registrar los cambios de estados

Consideraciones:

1. **Validez:** Este indicador es válido durante todo el proyecto cuando hay prueba en paralelo al desarrollo
2. **Proceso:** El indicador es muy útil para detectar el síndrome del 90%.

Indicador de Cobertura de Prueba

Muestra cuánto habría que probar, cuanto se puede probar y cuanto funciona bien.

Se realiza a partir de los estados de los casos de prueba

- Planificados
- Disponibles

- Ejecutados
- Ejecutados OK

Indicador de Earned Value

La idea es usar el avance calculado por funcionalidad completa para calcular el valor agregado

1. **Alcance:** Es necesario separar los costos que no están asociados directamente a la construcción de software.
2. **Validez:** Al igual que el indicador de Funcionalidad Completa, solo aplica a la construcción, no a la estabilización.
3. **Existe margen de error:** Los pesos generan información inexacta, etc.

Métricas Pruebas Unitarias

Ofrece una medida de los componentes que se han probado individualmente antes de la integración con respecto al total de componentes que han sido implementados.

$$\text{CPU} = \text{NPC} / \text{CImp}$$

CPU: Valor de la métrica para el control de pruebas de unidad.

NPC: Número de componentes probados individualmente antes de integrarlos

CImp: Número de componentes implementados.

Exhaustividad de la prueba

Amplitud de cobertura de prueba:

Indicador de cuantos requisitos se han probado del número total de estos.

$$\text{CP} = \text{CPE} / \text{CPR}$$

CP: Valor de cobertura de las pruebas

CPE: Número de casos de prueba que han sido ejecutados

CPR: Número de casos de prueba a ejecutar requeridos para cubrir todos los requerimientos.

Profundidad de la prueba:

Porcentaje de los caminos básicos independientes probados.

$$\text{PCB} = (\text{P} / \text{V}(\text{G})) \times 100$$

PCB: Porcentaje de caminos básicos

P: Número de pruebas diseñadas

V(G): Complejidad ciclóstima calculada anteriormente.

PDF: "Estrategias.pdf"

Estrategias de prueba

Se denomina estrategias de prueba a la organización de la prueba en una serie de pasos bien planificados que permiten una correcta construcción del software. Las estrategias de prueba sirven tanto para el desarrollo de software, como para el control de calidad grupo independiente de prueba y el cliente.

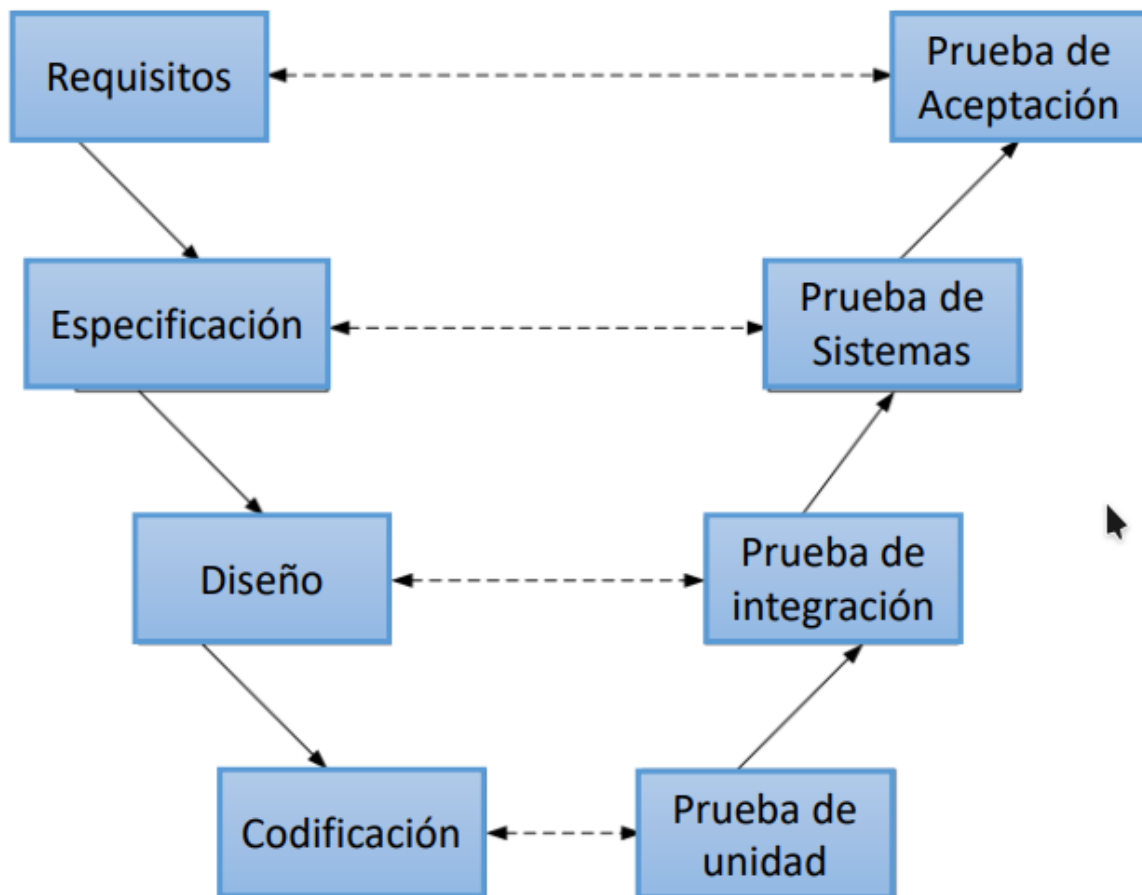
Las estrategias de prueba define la siguiente información:

- ¿Qué pasos realizar?, es decir, qué actividades llevar a cabo para realizar una prueba adecuada.
- ¿Cuándo se deben planificar?, o lo que es lo mismo, ¿cuándo, en el ciclo de desarrollo, se deben comenzar las actividades relacionadas con la organización de la prueba?.
- ¿Cuándo se deben realizar?, es decir, ¿Cuándo, en el ciclo de desarrollo, se deben realizar las actividades de prueba?
- ¿Cuánto esfuerzo, tiempo y recursos dedicar a la prueba?, definiendo su disponibilidad (ventana temporal), número y relacionándolo con las distintas actividades de la prueba.

Por lo tanto, la estrategia de pruebas incorpora:

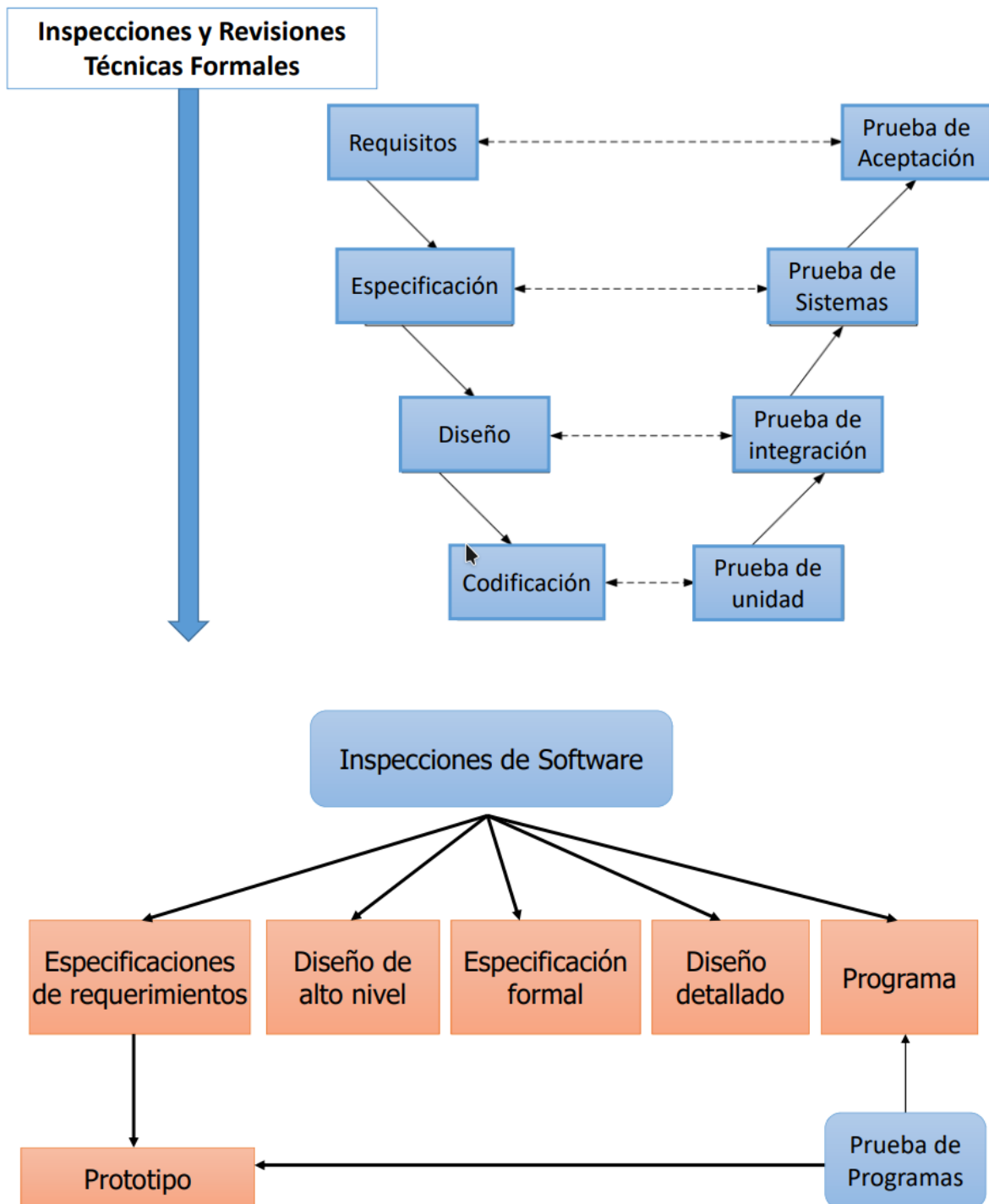
- La planificación de la prueba
- El diseño de los casos de prueba
- Ejecución
- Agrupación
- Evaluación de los datos resultantes de las actividades de prueba

Modelo en V



La prueba se corresponde con un conjunto de actividades que se pueden planificar por adelantado y llevar a cabo sistemáticamente.

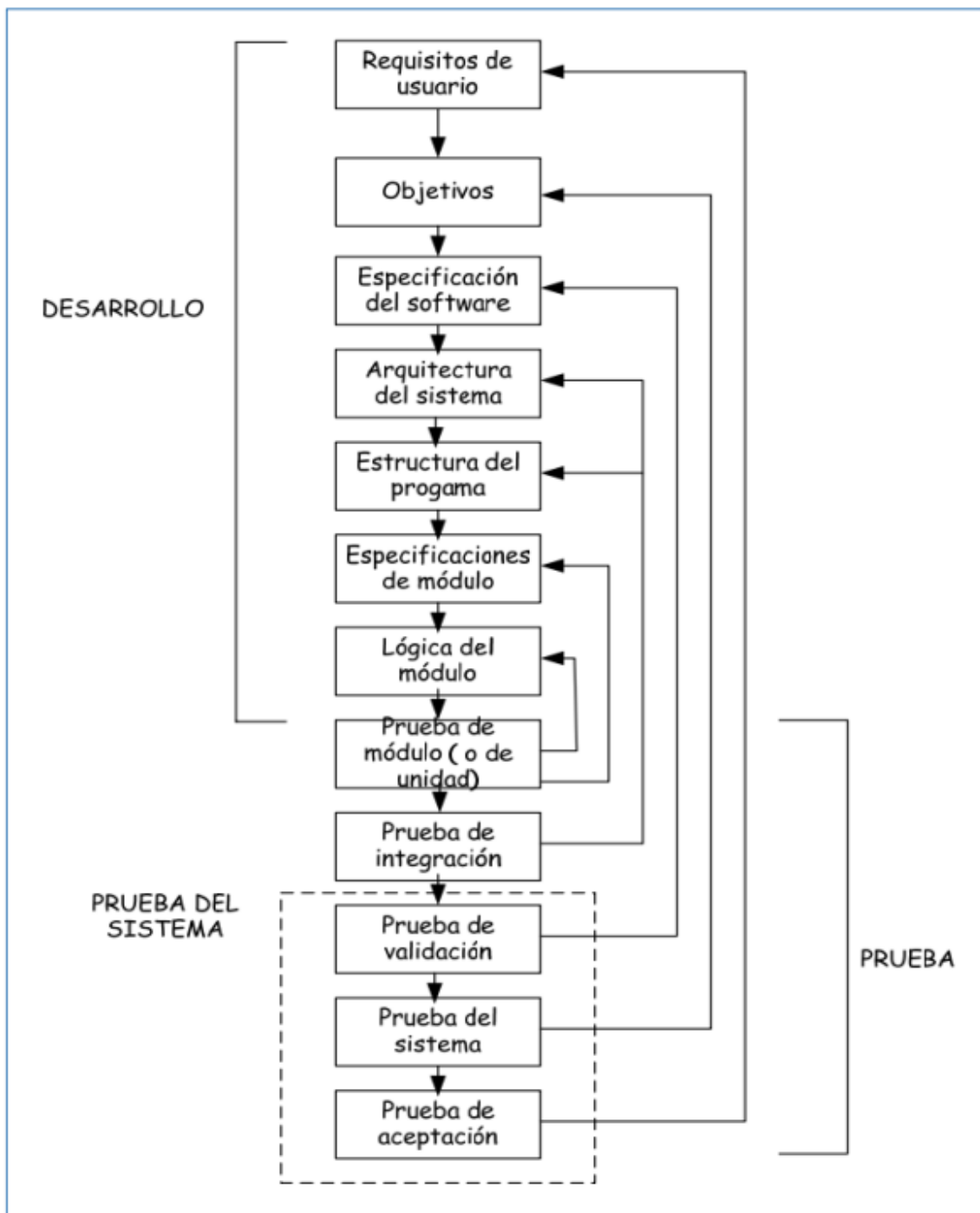
- El proceso de prueba es inverso al proceso de creación.
- En cada nivel en que se desarrolla la prueba son apropiadas solamente un subconjunto de las pruebas totales a realizar.
- Dependiendo del nivel y del tipo de organización, deberán realizar la prueba diferentes recursos (programadores, analistas, usuarios, grupo independiente de pruebas, control de calidad, etc.)
- Aunque la prueba y la depuración son actividades independientes, esta última se puede incluir en cualquier estrategia de pruebas.



Se pueden distinguir cinco fases en el proceso de prueba que **corresponden** secuencialmente con las **etapas genéricas** de las distintas **estrategias de prueba**:

- **Prueba de unidad.** Es la primera prueba realizada de todo el proceso de pruebas y corresponde con la prueba de cada módulo del programa.

- **Prueba de integración.** Se inicia una actividad consistente en integrar los módulos con el fin de probar sus interfaces, contando con la información de diseño como ayuda.
- **Prueba de validación.** Una vez ensamblado el software se comprueba que cumple las especificaciones. Para ello será imprescindible.
- **Prueba de sistema.** Consiste en que el software, una vez validado, se relacione con los restantes elementos que conforman el sistema global.
- **Prueba de aceptación.** El último paso antes de la entrega formal de software al cliente y se realiza, normalmente, en entornos de usuarios.



El ciclo de vida del testing



Pasos básicos:

- Planificación y control
- Análisis y diseño
- La implementación y ejecución
- La evaluación de los criterios de salida y presentación de informes
- Las actividades de cierre

Planificación

- Entendemos los objetivos del cliente, y los riesgos que se abordan con las pruebas.
- Las tareas de planificación nos ayudan a construir el plan de pruebas.
- Determinar el alcance y los riesgos e identificar los objetivos de la prueba:
 - Tenemos en cuenta el software, los componentes, otros sistemas o productos que están en el ámbito de las pruebas, los riesgos que

deben ser abordados, y para qué estamos probando: para descubrir defectos, para demostrar que el software cumple con los requisitos, para demostrar que el sistema es adecuado para el propósito o para medir las cualidades y atributos de software.

- Determinar el método de prueba:
 - Considerando cómo vamos a llevar a cabo las pruebas, las técnicas a utilizar, quién debe involucrarse y cuando, lo que vamos a producir como parte de las pruebas.
- Poner en práctica la política de prueba y/o la estrategia de prueba.
 - Debemos asegurarnos de que lo que vamos a hacer se adhiere a la política y la estrategia establecida en la organización.
- Determinar los recursos de prueba necesarios
 - Decidimos el equipo de prueba, también establecemos todo el hardware.
- Programar el análisis de la prueba y las tareas de diseño, ejecución de pruebas, y la evaluación.
 - Colocar dentro del plan del proyecto el tiempo necesario para todas las actividades de prueba.
- Determinar los criterios de salida:
 - Criterio de cobertura (por ejemplo, el porcentaje de declaraciones en el programa que se debe ejecutar durante la prueba) que ayudará a llevar un registro para determinar si estamos complementando las actividades de prueba correctamente.
 - Los criterios muestran las tareas y los controles que debemos completar en las pruebas antes de poder decir que la prueba finalizó.

Diseño

- Es la actividad en la que los objetivos generales de prueba se transforman en condiciones de prueba tangibles y en diseños de casos de prueba.
- Revisar la base de pruebas: Revisar los documentos de análisis, arquitectura, riesgos, especificaciones.
 - Se pueden encontrar algunos errores no detectados.
 - Evitar que lleguen al código.
 - Se pueden bosquejar pruebas de caja negra.
- Diseñar entornos de prueba:

- Identificar la infraestructura y las herramientas necesarias
 - Hojas de cálculo
 - Procesadores de texto
 - Herramientas de planificación de proyectos
 - Herramientas de pruebas
 - Y todo lo necesario para llevar a cabo el trabajo de prueba

Ejecución

- Implementación y ejecución de las pruebas
- Implementación:
 - Desarrollar y priorizar los casos de prueba
 - Creación de conjunto de pruebas a partir de los casos de prueba para su ejecución
 - Implementar el medio ambiente
- Ejecución:
 - Ejecutar los casos de prueba
 - Registrar el resultado de la ejecución
 - Comparar los resultados reales con los esperados
 - Elaborar un informe de discrepancias
 - Repetir las actividades de prueba como consecuencia de las discrepancias

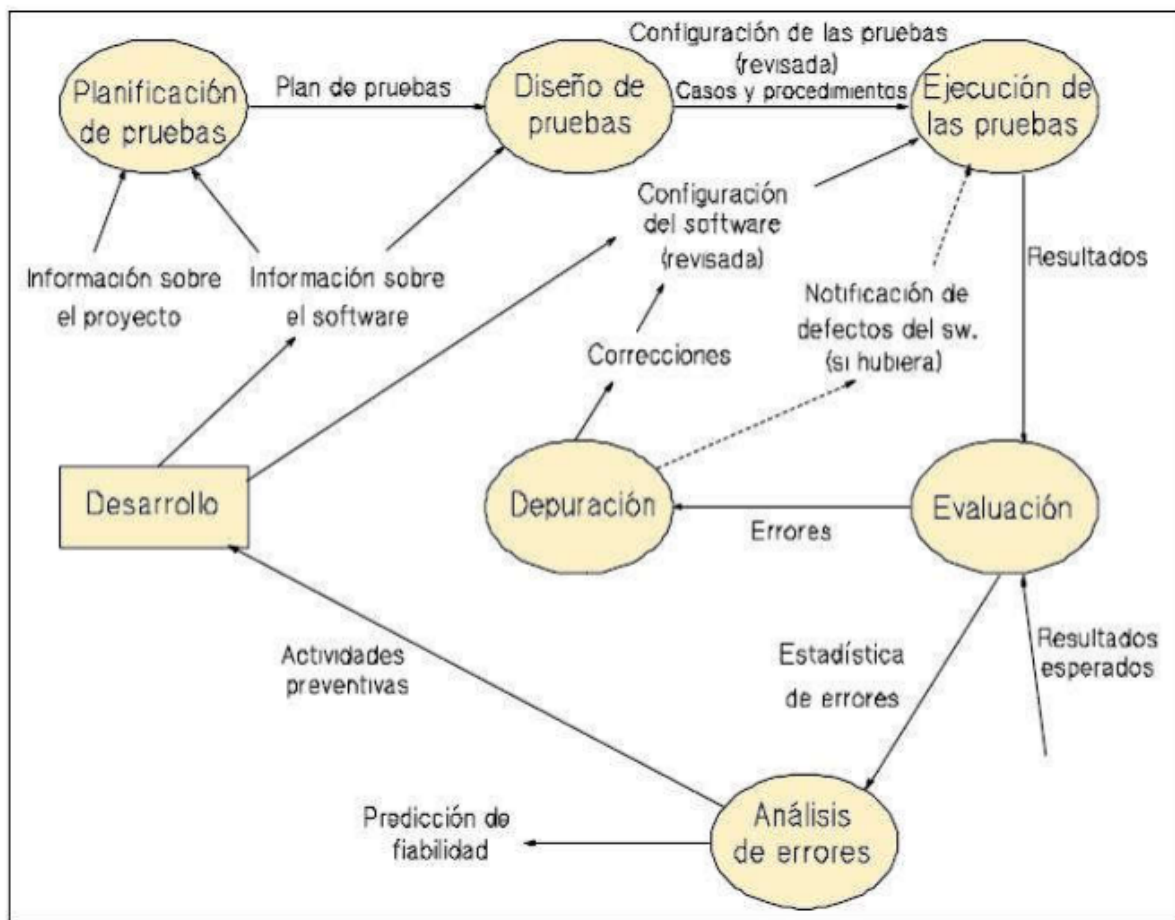
Evaluación

- Evaluación de los criterios de fin de prueba
- Se hace para cada tipo nivel de prueba, para determinar si se ha hecho lo suficiente para encontrar errores
- La suficiencia lo determinan los criterios de salida de prueba
- Estos dependen de las características de la aplicación
- Estos criterios deben ser evaluados:
 - Comprobar con los registros de prueba con los criterios de salida de prueba
 - Evaluar si es necesario realizar más pruebas o si los criterios de prueba no eran correctos

Fin

- Actividades de cierre de las pruebas
- Recopilación de datos
- Verificar que todos los incidentes se hayan corregido
- Documentar todos los aspectos del proceso de prueba para su reutilización
- Entregar toda la documentación, software, etc... al área de soporte
- Evaluar el proceso de prueba para proyectos futuros

Ciclo de vida del testing



Plan de Pruebas

- La documentación de la prueba de software es el elemento vital que eleva las actividades experimentales al nivel de una prueba de software.

- IEEE 829 Estándar 839 para la documentación de prueba de software y sistema
- El objetivo del estándar es proporcionar un conjunto estandarizado de documentos para la documentación de pruebas de software.

El Plan de pruebas es el resultado de la planificación de las pruebas, en términos generales este plan incluye:

- El objetivo del proceso de prueba.
- Los objetos que hay que probar.
- Las características que se van a probar y las que no.
- El método de prueba a utilizar.
- Los recursos que se van a emplear.
- El plan de tiempos.
- Los productos a generar durante las pruebas
- El reparto de las responsabilidades

Introducción

Propósito

- Finalidad del plan de pruebas, que sistema se aplicará, se utiliza algún estándar.
- Objetivos del proyecto
- Participantes
- Descripción de la metodología utilizada.

Ámbito del proyecto

- Objetivos de Pruebas

Tipos de Pruebas

Revisión

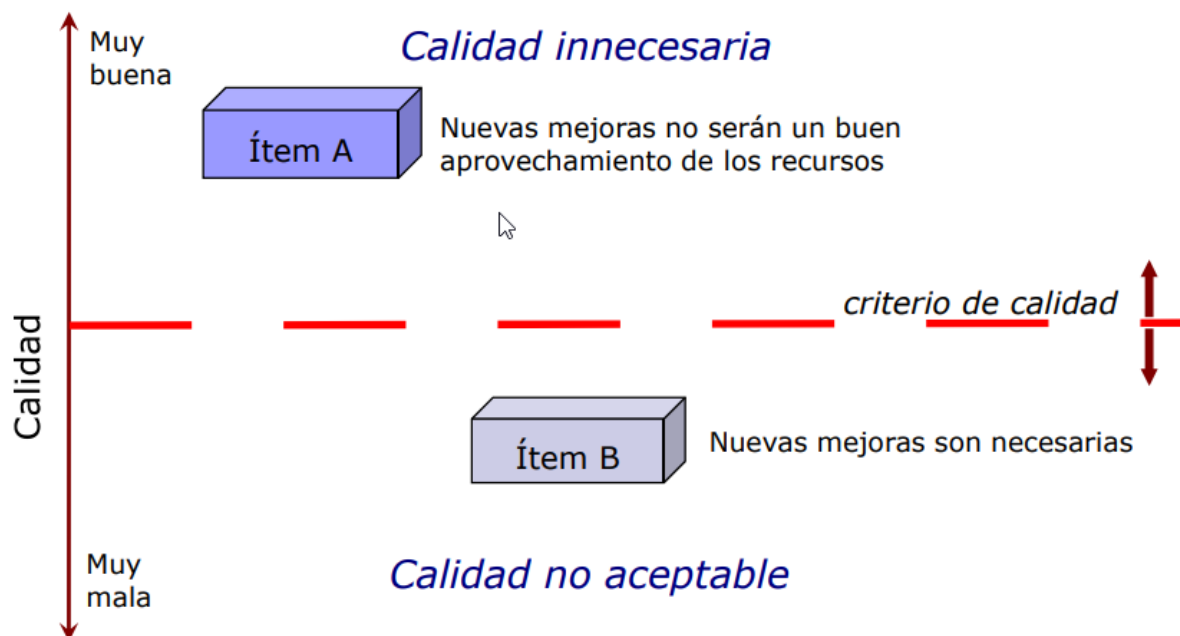
- Hitos de fases, artefactos o entregables disponibles que se le aplicará las revisiones.
- Tipo de revisión a cada fase/entregable
- Criterios de terminación de revisiones
- Recursos, calendario
- Herramientas

Pruebas

- Tipos de pruebas

- Unitarias, integración, **validación**, sistemas.
- Objetivos de cada tipo de prueba.
- Criterios de finalización.
 - Ejecución de casos de prueba seleccionados
 - Porcentaje de cobertura de casos alcanzado
 - Porcentaje de efectos detectados por unidad de tiempo bajo un valor dado
 - Cantidad de defectos remanente por severidad
 - Cantidad de defectos detectados
 - Tiempo de testing transcurrido

Plan de Pruebas: Criterios de finalización



Criterios de validación.

Para liberar exitosamente un producto se requiere:

- Todos los componentes, incluida documentación de usuario, completos y probados
- Políticas definidas de liberación y soporte
- Reproduccion / Distribucion disponibles
- Soporte disponible
- Clientes / Usuarios saben cómo reportar problemas

Estrategias de prueba

Explicación de cómo se realizarán las actividades descritas en los puntos anteriores.

- Procedimiento para las revisiones
 - Detalle de cómo se realizará el procedimiento de las revisiones quienes participan, como se registra el resultado, que checklist se utilizaran. Responsables de la actividad para cada uno de los entregables/fases que se planificó realizar la revisión.
- Procedimiento para las pruebas
 - Procedimiento describiendo: objetivos de la prueba, técnica utilizada, criterios de finalización. Para cada tipo de prueba planteada.

Catálogo de defectos

Tipo de defectos:

- Crítico: El defecto impide acceder a la funcionalidad. El defecto impide continuar con la operación del sistema al no existir un camino alternativo
- Severo: Es posible acceder a la funcionalidad pero la misma no puede ser ejecutada en forma completa. Existe una severa desviación respecto a los requerimientos. El defecto restringe la ejecución de gran parte de la funcionalidad definida aunque exista un camino alternativo de solución.
- Normal: Es posible acceder a la funcionalidad, esta no se ejecuta completamente o no cumple con el resultado esperado por defectos típicos en el desarrollo del software.
- Trivial: El defecto no tiene impacto en el resultado o proceso respecto a los requerimientos solicitados por el cliente.

Catálogo de defectos extendido:

1. Leve
2. Moderado
3. Incómodo
4. Trastorno
5. Serio
6. Muy serio
7. Extremo
8. Intolerable

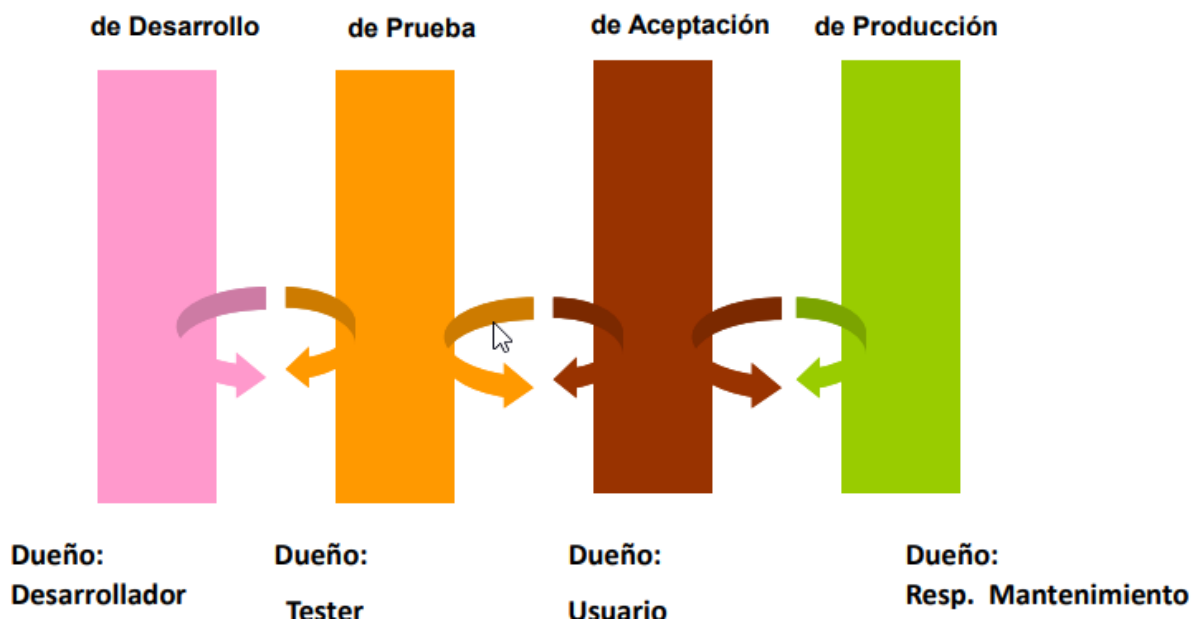
9. Catastrófico

10. Infeccioso

Entregables

- Anexos u otros documentos donde se detalle lo siguiente:
 - Detalle de los casos de prueba a ejecutar para cada nivel de prueba
 - Informe de defectos
 - Informe final de pruebas.

Ambientes de Prueba



Administración de Ambientes

- Los ambientes de **desarrollo** son inestables y en general los desarrolladores instalan lo que desean.
- El ambiente de **Testing** debe ser lo más parecido posible al de producción. Tiene que ser controlado.
- El ambiente de **Aceptación**, tiene que ser lo más parecido posible al de producción y tiene que ser controlado.

- El ambiente de **producción** es un ambiente. Debe instalarse productos solo con autorización y con las pruebas en el ambiente de aceptación aprobadas.